

Final Exam

Nishant Khandhar A20581012

2025-12-09

Instructions And Rules

1. Insert your name and A# in the author field above. You should only be taking this version of the exam if the *second to last digit* of your A# is *odd* (e.g. 1, 3, 5, 7, or 9).
2. You may use both textbooks as well as any course notes.
3. You may not consult ChatGPT or any other LLM-based chatbot. If you are caught violating this rule, you will be subject to failure on this exam and potentially the course.
4. You may not consult any other person (e.g. a classmate or your buddy who took this course last semester). If you are caught violating this rule, you will be subject to failure on this exam and potentially the course.
5. Please answer all of the questions below. You will need to write code in the appropriate code blocks. You will need to write text to answer the questions.
6. Please try to differentiate my questions from your answers in a clean and readable way. Please write in reasonably complete and comprehensible sentences.
7. You can add whatever libraries you need to the imports section below, but try to limit it to libraries that you're actually using. I believe the ones listed there are sufficient, but you may use whatever you want.
8. Your Rmd file *must knit* at the end. You will be downgraded if this does not happen.

Regression

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.1.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x tidyr::expand() masks Matrix::expand()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## x tidyr::pack() masks Matrix::pack()
## x tidyr::unpack() masks Matrix::unpack()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(ggrepel)
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(gam)
```

```
## Loading required package: splines
## Loading required package: foreach
##
## Attaching package: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##   accumulate, when
##
## Loaded gam 1.22-6
```

The goal of your final is to fit a regression model. The data is loaded below. (Make sure to put your data in the same path as this Rmd file.)

```
data <- read.table("final_data_b.csv", sep=";", header=TRUE)
```

This is taken from a housing sales dataset. Here is a brief dictionary for the features:

Lot.Frontage: The length of the lot along the road or street **Lot.Area:** Total area of the lot **Neighborhood:** Neighborhood in which the property is located **Year.Built:** Year the structure was built **Garage.Type:** Garage location

2Types More than one type of garage
Attchd Attached to home

Basement Basement Garage
 BuiltIn Built-In (Garage part of house - typically has room above garage)
 CarPort Car Port
 Detchd Detached from home
 NA No Garage

Garage.Cars: Number of cars that fit in the garage

Garage.Finish: Interior finish of the garage

Fin Finished
 RFn Rough Finished
 Unf Unfinished
 NA No Garage

Paved.Drive: Paved Drive (Ordinal): Paved driveway

Y Paved
 P Partial Pavement
 N Dirt/Gravel

X1st.Flr.SF: First floor square footage

X2nd.Flr.SF: Second floor square footage

Full.Bath: Number of full bathrooms

Bedroom.AbvGr: Number of bedrooms above ground

SalePrice: The most recent Sale price

Your goal will be to predict the sale price using the other features. You *must* fill in the code blocks given, but if you want to add extra plots or derive additional features, it is up to you to present the best model you can.

Exploration

Below, show the number of missing values per column:

```
missing_counts <- colSums(is.na(data))
missing_counts
```

```
## Lot.Frontage    Lot.Area Neighborhood Year.Built Garage.Type
##           171           0           0           0           0
## Garage.Finish  Garage.Cars   Paved.Drive   Full.Bath  X1st.Flr.SF
##           0           0           0           0           0
##  X2nd.Flr.SF Bedroom.AbvGr   SalePrice
##           0           0           0
```

Show frequency counts of the various categorical features below (you can decide which features count as categorical):

```
# ---- freq_counts ----
```

```
tbl_neighborhood <- table(data$Neighborhood)
tbl_garage_type <- table(data$Garage.Type)
tbl_garage_finish <- table(data$Garage.Finish)
tbl_paved_drive <- table(data$Paved.Drive)
```

```
tbl_neighborhood
```

```
##
## Blmngtn BrDale BrkSide ClearCr CollgCr Crawfor Edwards Gilbert Greens GrnHill
##      6      8      38      15      85      33      63      66      3      2
## IDOTRR MeadowV Mitchel  NAMES NoRidge NPkVill NridgHt  NWAmes OldTown  Sawyer
##     38     11     43     162     19      6      57     47     83     54
## SawyerW Somerst StoneBr  SWISU  Timber Veenker
##     37     63     22     12     25      2
```

```
tbl_garage_type
```

```
##
##      2Types Attchd Basment BuiltIn CarPort Detchd
##     58     12     593     12     53      5     267
```

```
tbl_garage_finish
```

```
##
##      Fin RFn Unf
##    58 261 263 418
```

```
tbl_paved_drive
```

```
##
##      N      P      Y
##    82    19   899
```

Plot the numerical features (including sales price) below as histograms:

```
# ---- plot_hist ----
```

```
library(ggplot2)
```

```
num_cols <- c(
  "Lot.Frontage", "Lot.Area", "Year.Built", "Garage.Cars",
  "X1st.Flr.SF", "X2nd.Flr.SF", "Full.Bath",
  "Bedroom.AbvGr", "SalePrice"
)
```

```
for (col in num_cols) {
  plt <- ggplot(data, aes_string(x = col)) +
    geom_histogram(bins = 30, fill = "orange", color = "white", alpha = 0.7) +
    labs(title = paste("Histogram of", col), x = col, y = "Count") +

```

```

    theme_minimal()

    print(plt)
}

```

```

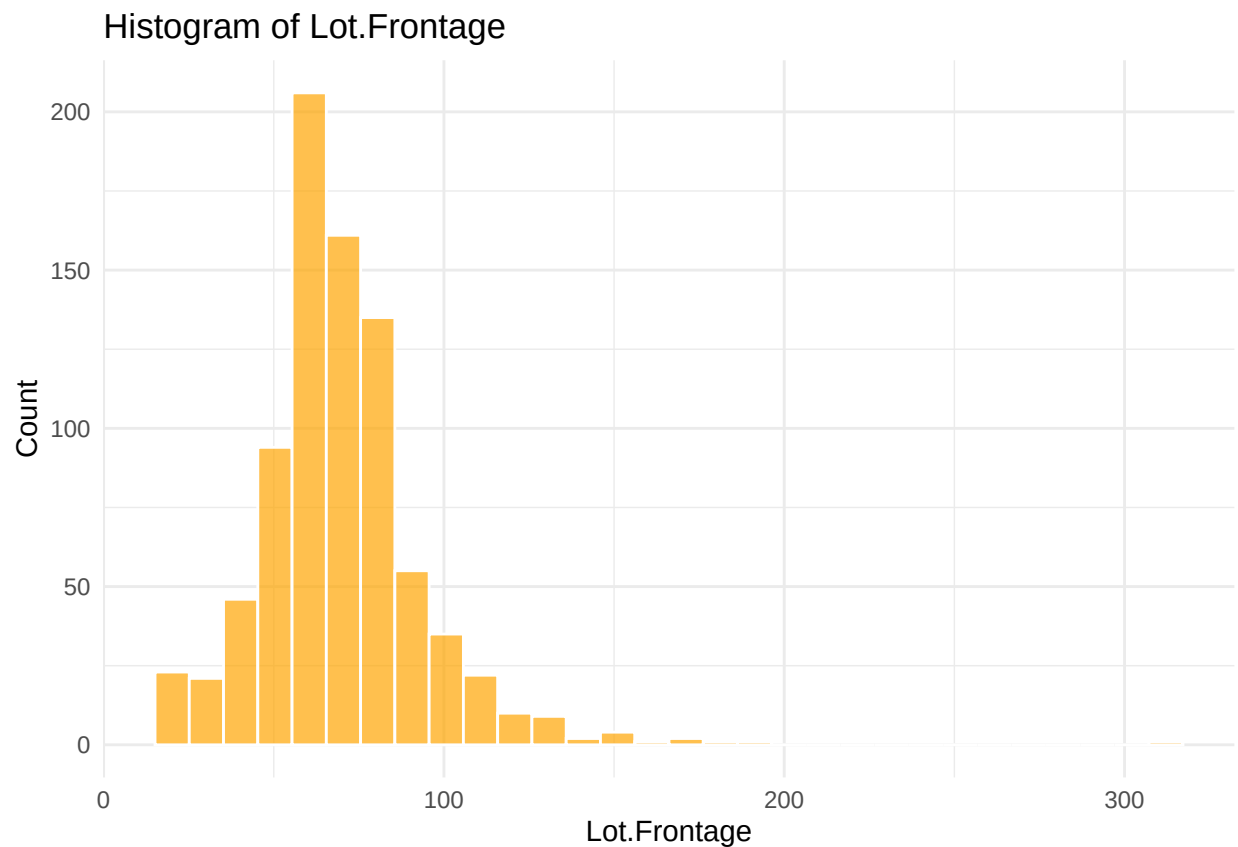
## Warning: 'aes_string()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'.
## i See also 'vignette("ggplot2-in-packages")' for more information.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

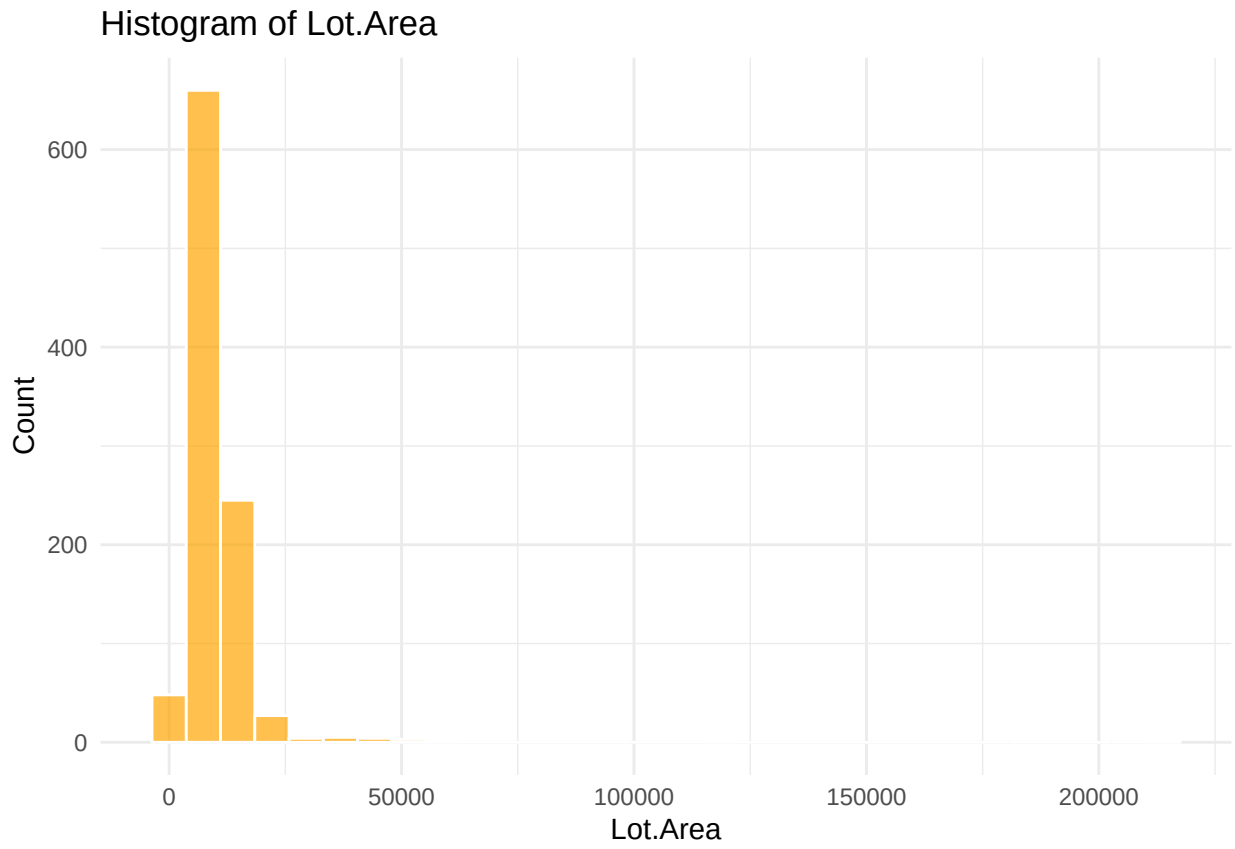
```

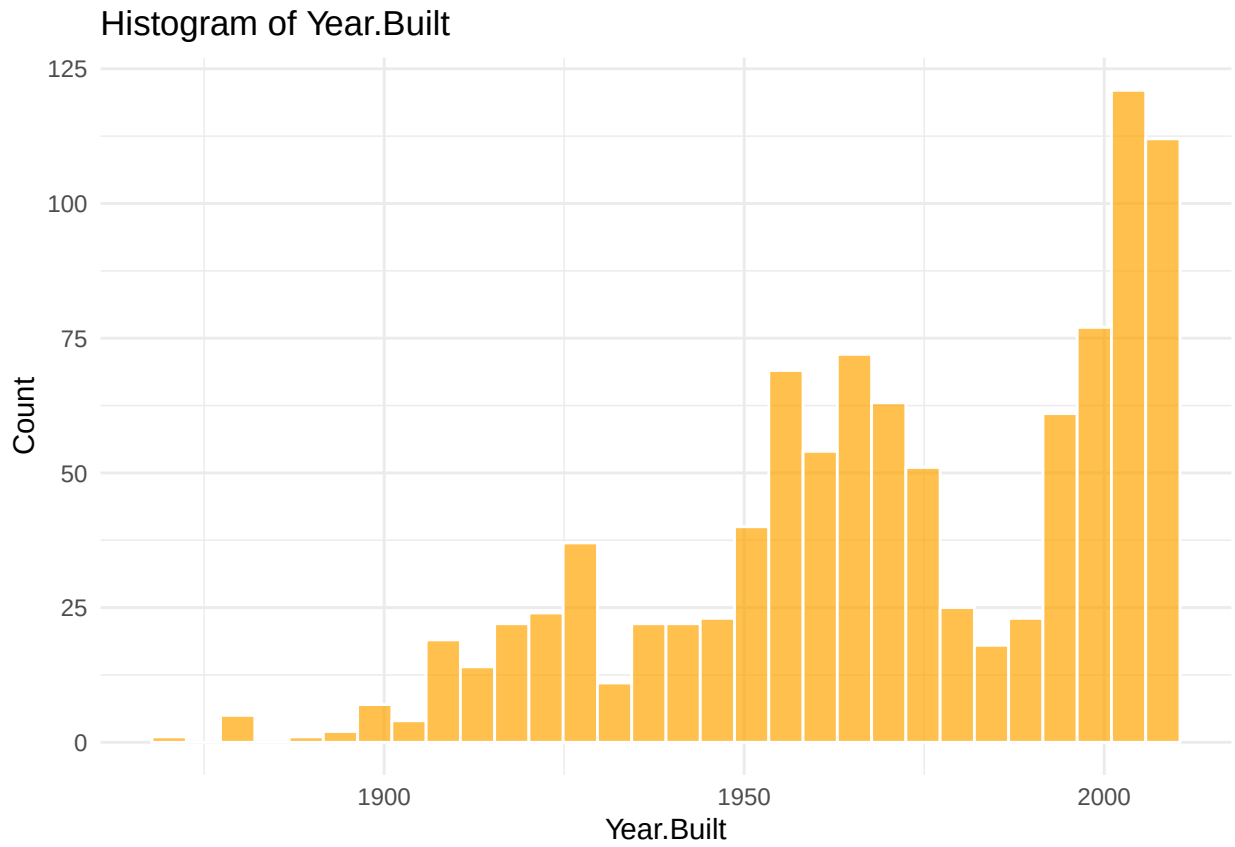
```

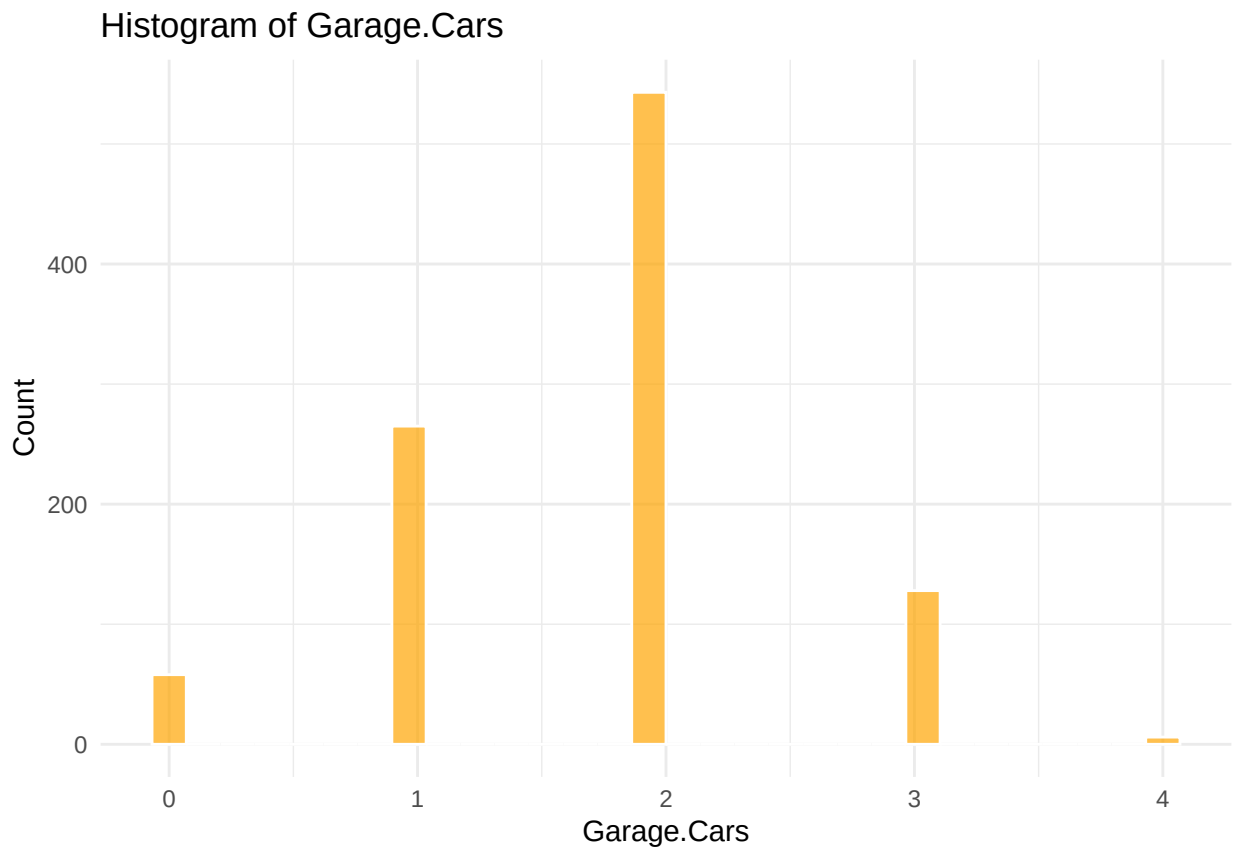
## Warning: Removed 171 rows containing non-finite outside the scale range
## ('stat_bin()').

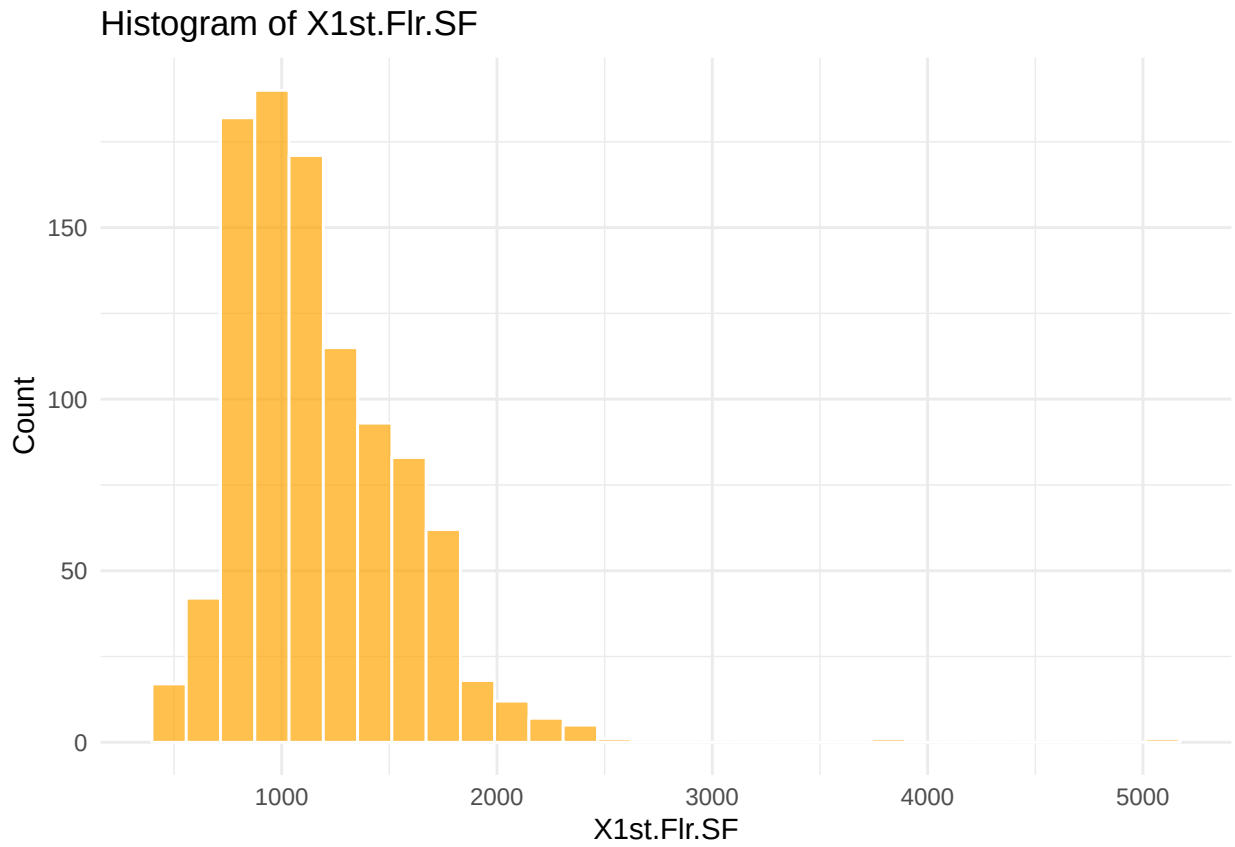
```

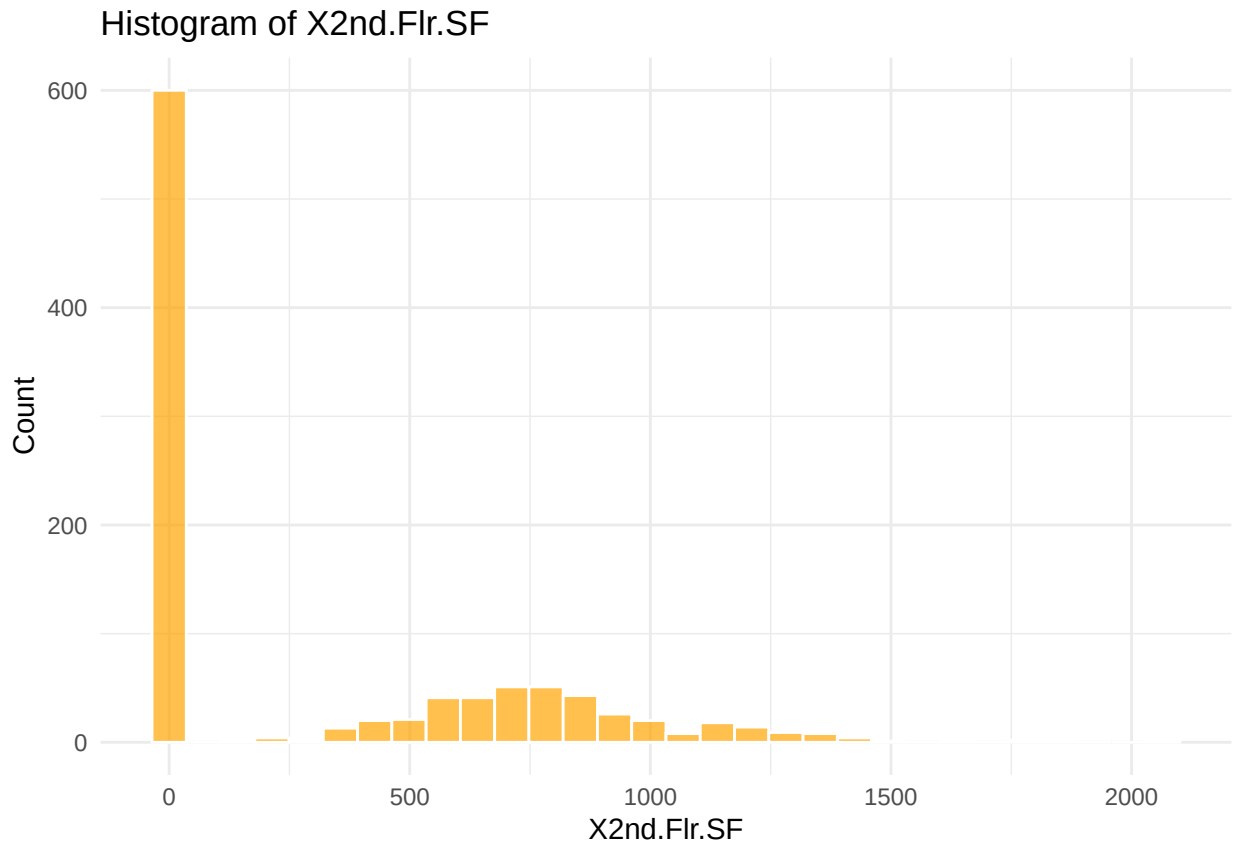


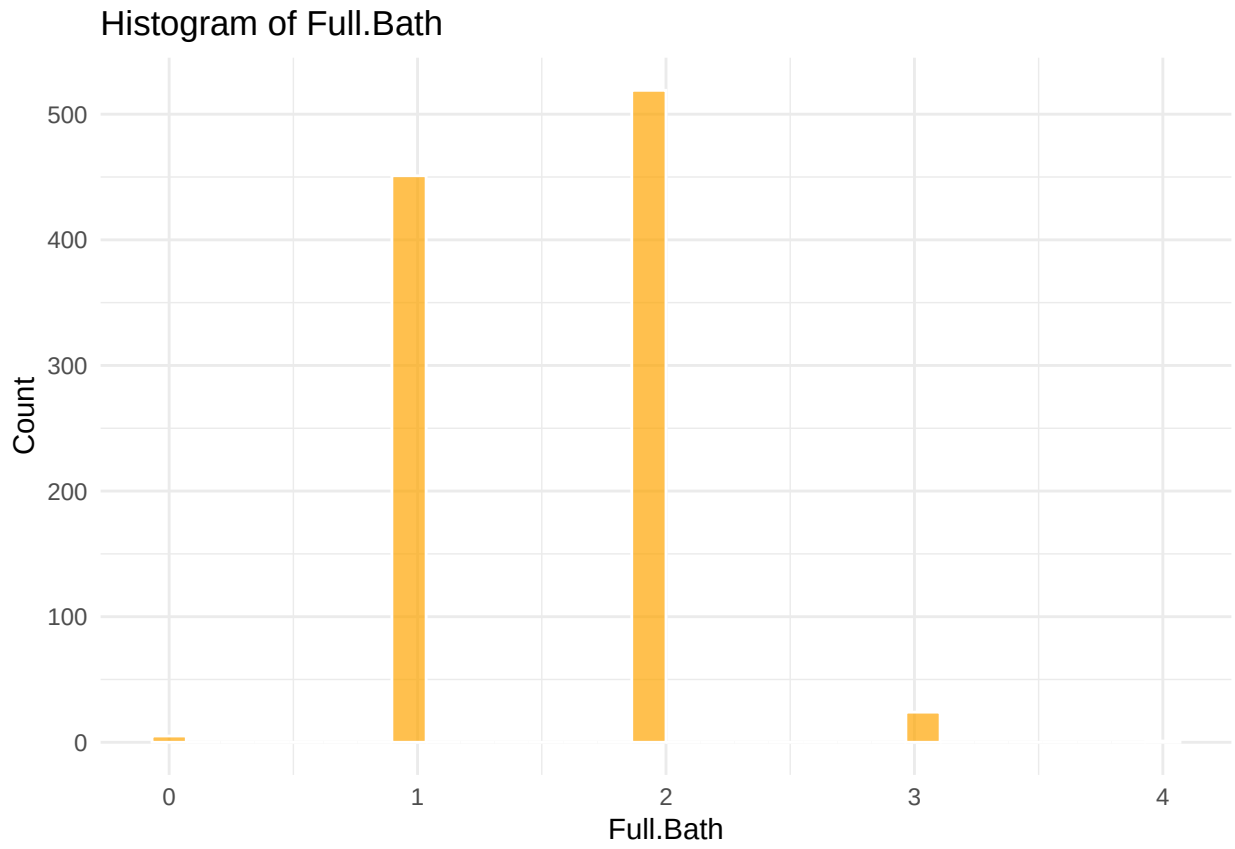


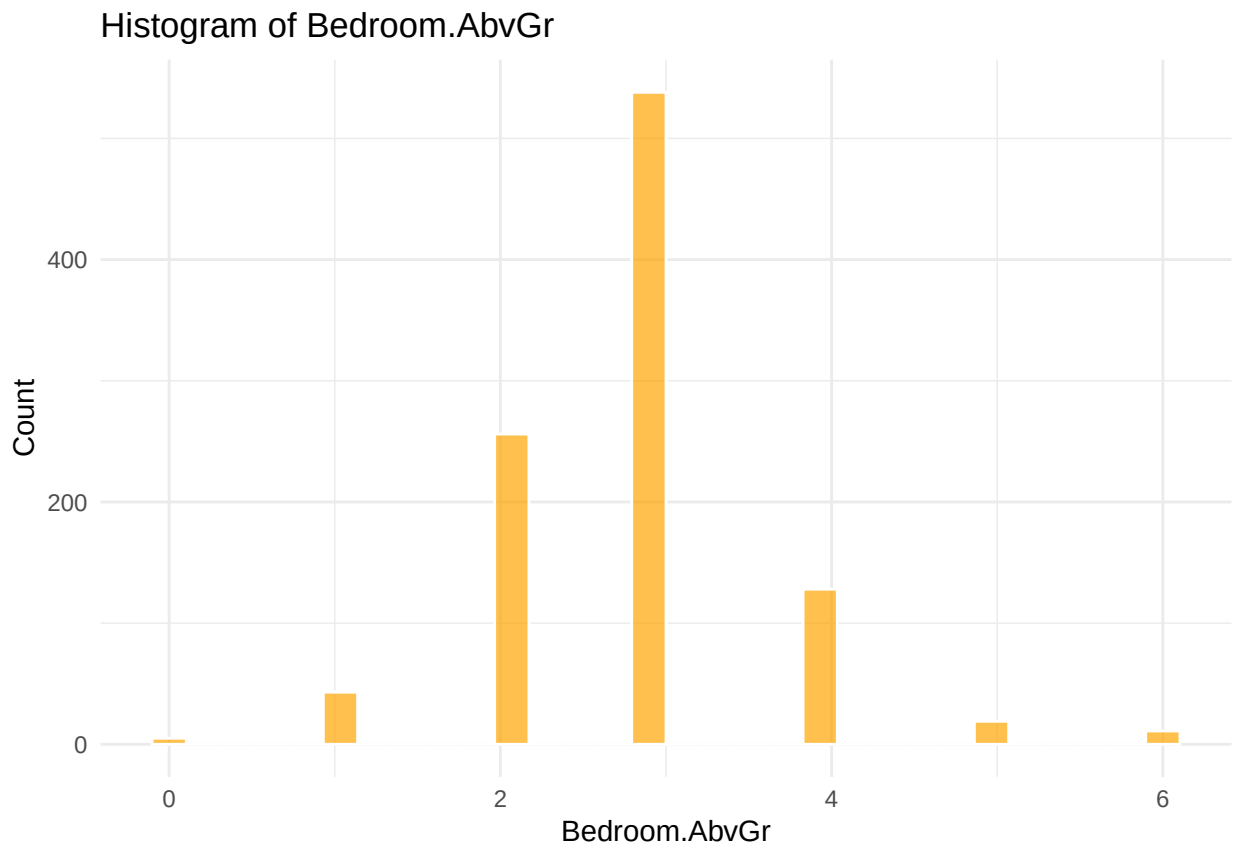


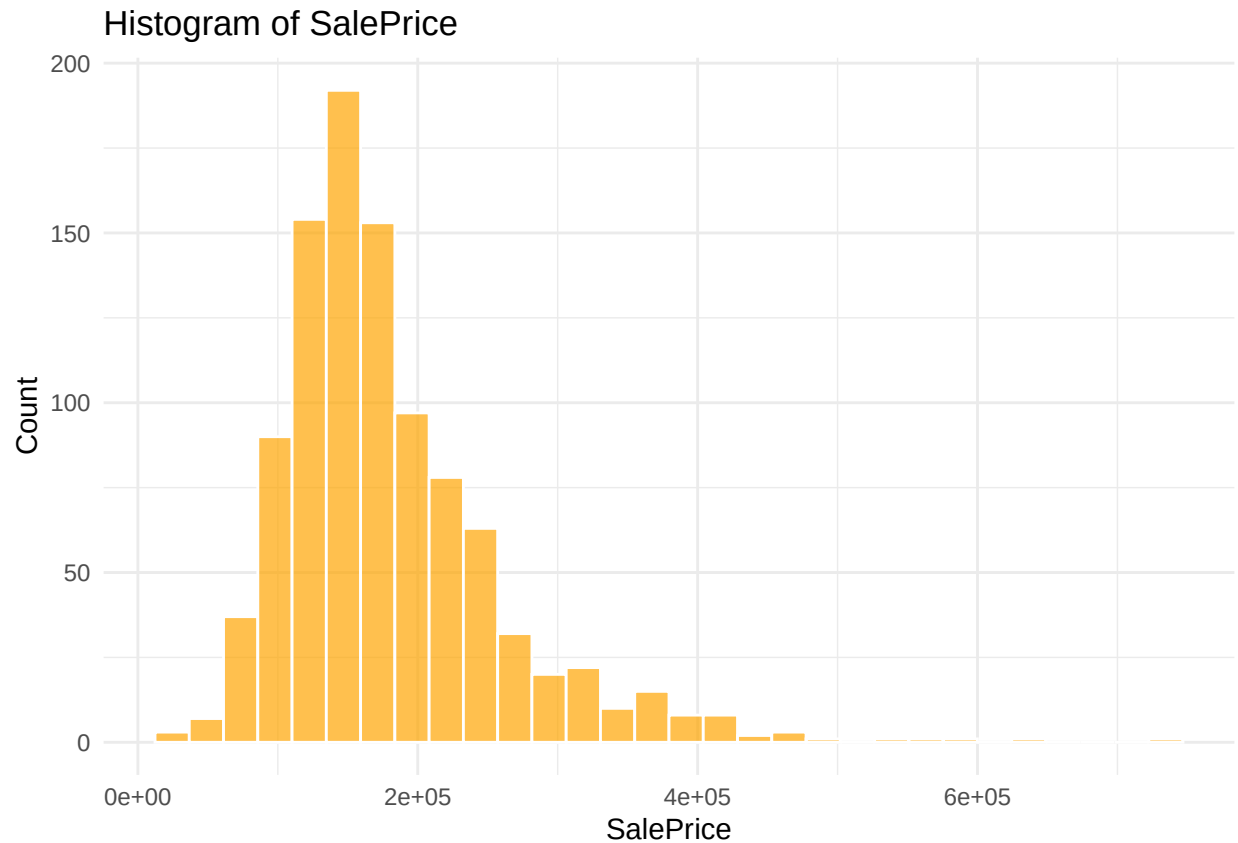












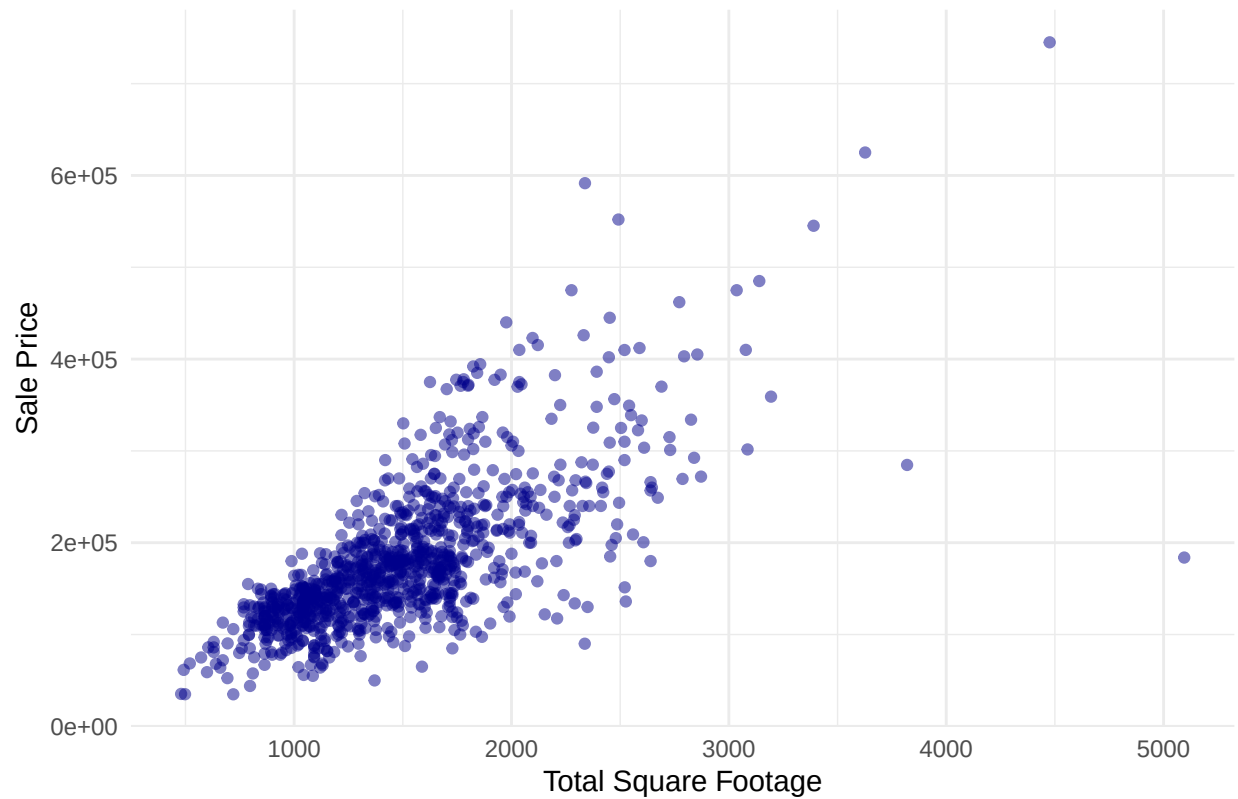
Provide any other plots or exploratory analysis you think will be enlightening below:

```
# ---- other_plots ----

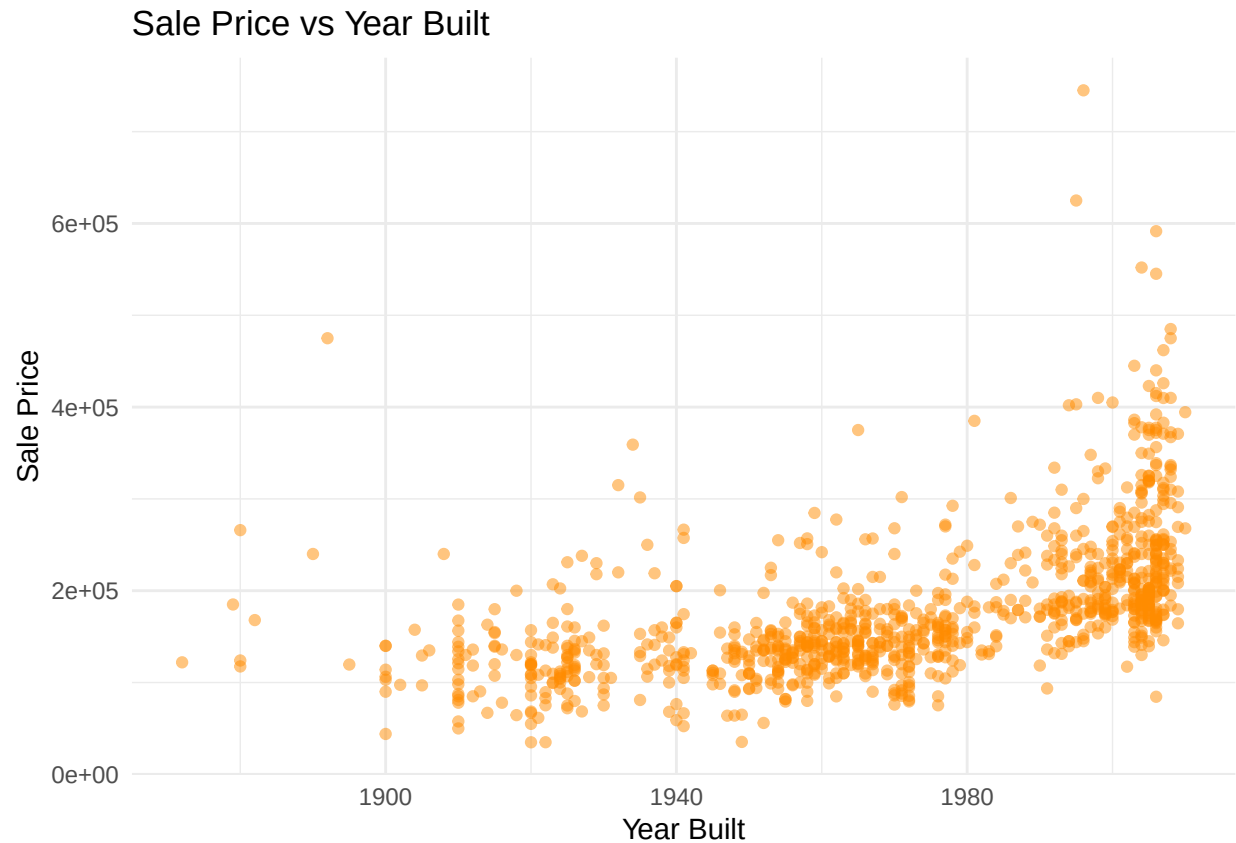
# Total square footage
data$TotalSF <- data$X1st.Flr.SF + data$X2nd.Flr.SF

# SalePrice vs Total SF
ggplot(data, aes(x = TotalSF, y = SalePrice)) +
  geom_point(alpha = 0.5, color = "darkblue") +
  labs(
    title = "Sale Price vs Total Square Footage",
    x = "Total Square Footage",
    y = "Sale Price"
  ) +
  theme_minimal()
```

Sale Price vs Total Square Footage



```
# SalePrice vs Year Built
ggplot(data, aes(x = Year.Built, y = SalePrice)) +
  geom_point(alpha = 0.5, color = "darkorange") +
  labs(
    title = "Sale Price vs Year Built",
    x = "Year Built",
    y = "Sale Price"
  ) +
  theme_minimal()
```



```
# ---- other_analysis ----
```

```
library(reshape2)
```

```
##
```

```
## Attaching package: 'reshape2'
```

```
## The following object is masked from 'package:tidyr':
```

```
##
```

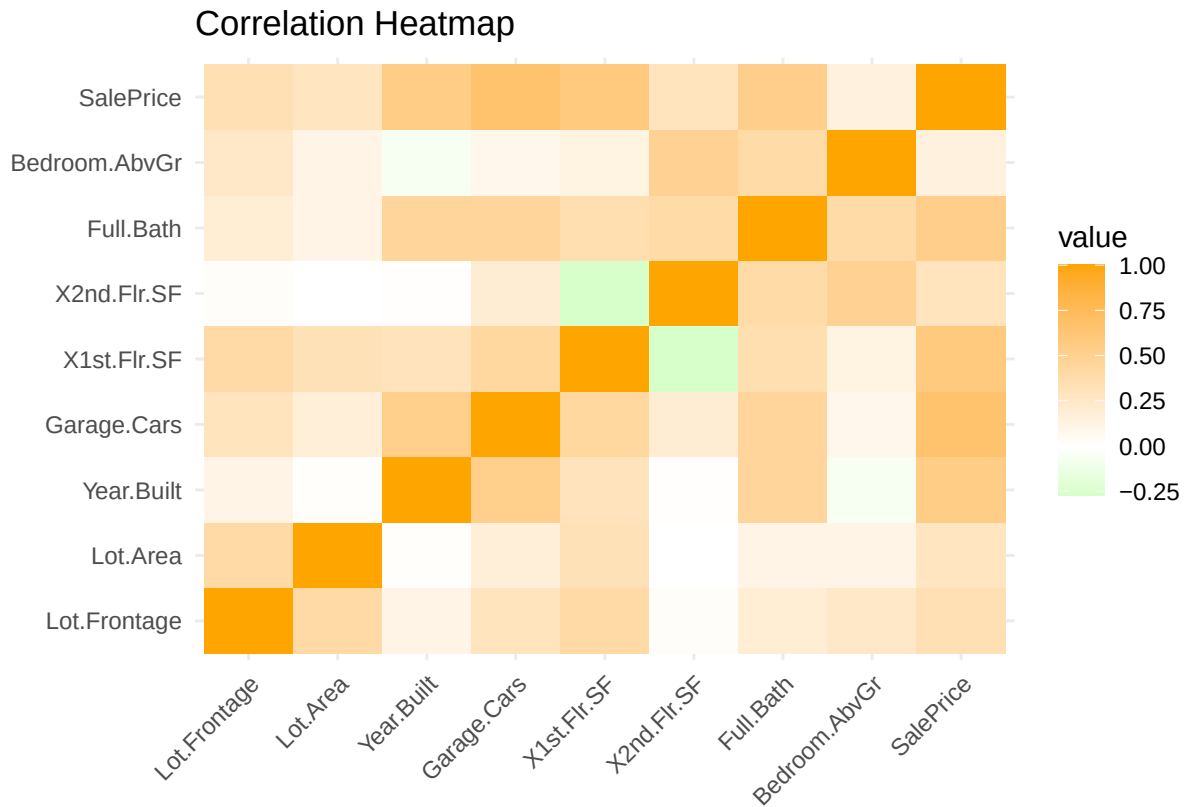
```
## smiths
```

```
num_df <- data[, num_cols]
```

```
corrmat <- cor(num_df, use = "pairwise.complete.obs")
```

```
melted_corr <- melt(corrmat)
```

```
ggplot(melted_corr, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "green", high = "orange", mid = "white") +
  labs(title = "Correlation Heatmap", x = "", y = "") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Can you find some outlying behavior? Should you eliminate these or keep them?

Answer

A few properties with unusually large lot areas, very high total square footage, or unusually high sale prices are displayed in the scatter plots and histograms. Clearly, these points differ from the majority of the information.

But unless these observations are the result of data entry errors, they are not noise but rather real high-value homes. The model would be distorted if expensive homes were excluded because housing prices naturally vary widely.

Because they represent actual fluctuations in the housing market and could enhance the model's capacity to forecast upscale properties, these outliers ought to be retained.

Feature Transformations

You need to decide what to do with the missing values in `Lot.Frontage`. Include code below to handle these:

```
# ---- handle_missings_frontage ----
library(dplyr)

overall_med_frontage <- median(data$Lot.Frontage, na.rm = TRUE)

data <- data %>%
```

```
group_by(Neighborhood) %>%
mutate(
  Lot.Frontage = ifelse(
    is.na(Lot.Frontage),
    ifelse(
      all(is.na(Lot.Frontage)),
      overall_med_frontage,
      median(Lot.Frontage, na.rm = TRUE)
    ),
    Lot.Frontage
  )
) %>%
ungroup()
```

You should add a column to indicate if there is no garage (hint: the **Garage** features take the empty string value when the value is missing):

```
# ---- no_garage ----
data <- data %>%
  mutate(
    NoGarage = as.integer(is.na(Garage.Type) | Garage.Type == "")
  )
```

You may want a feature indicating the total square footage of the house (combined 1st and 2nd floor):

```
# ---- square_feet ----
data <- data %>%
  mutate(TotalSF = rowSums(across(c(X1st.Flr.SF, X2nd.Flr.SF))))
```

If there are any other transformations you would like to explore, you may place these below:

GAMs

(See, for example, the Chapter 7 lab.)

Split your data into a training set (800 samples) and test (200 samples).

```
# ---- train_test_split ----
set.seed(123)

train_idx <- sample(seq_len(nrow(data)), size = 800)

train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]
```

Fit a basic linear regression on the training data:

```
# ---- basic_lm ----

basic_lm <- lm(
  SalePrice ~ TotalSF + Year.Built + Garage.Cars + Full.Bath +
    Bedroom.AbvGr + Lot.Area + Lot.Frontage +
```

```

    Neighborhood + Garage.Type + Paved.Drive + NoGarage,
    data = train_data
)

summary(basic_lm)

```

```

##
## Call:
## lm(formula = SalePrice ~ TotalSF + Year.Built + Garage.Cars +
##     Full.Bath + Bedroom.AbvGr + Lot.Area + Lot.Frontage + Neighborhood +
##     Garage.Type + Paved.Drive + NoGarage, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -323351  -17076   -1035   16151  239877
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.203e+06  2.132e+05  -5.645  2.34e-08 ***
## TotalSF         7.157e+01  4.569e+00  15.667  < 2e-16 ***
## Year.Built      6.253e+02  1.072e+02   5.835  7.97e-09 ***
## Garage.Cars     1.887e+04  3.173e+03   5.946  4.20e-09 ***
## Full.Bath       7.989e+02  3.641e+03   0.219  0.826374
## Bedroom.AbvGr  -9.435e+03  2.101e+03  -4.490  8.23e-06 ***
## Lot.Area        9.506e-01  1.522e-01   6.246  7.00e-10 ***
## Lot.Frontage    1.308e+02  7.222e+01   1.811  0.070555 .
## NeighborhoodBrDale  4.525e+03  2.178e+04   0.208  0.835456
## NeighborhoodBrkSide  3.429e+04  1.931e+04   1.776  0.076094 .
## NeighborhoodClearCr  2.621e+04  2.029e+04   1.292  0.196753
## NeighborhoodCollgCr  2.396e+04  1.740e+04   1.377  0.169033
## NeighborhoodCrawfor  7.042e+04  1.888e+04   3.730  0.000206 ***
## NeighborhoodEdwards  5.608e+03  1.816e+04   0.309  0.757517
## NeighborhoodGilbert  3.957e+03  1.762e+04   0.225  0.822368
## NeighborhoodGreens   1.949e+04  4.057e+04   0.480  0.631046
## NeighborhoodGrnHill  1.230e+05  3.122e+04   3.942  8.84e-05 ***
## NeighborhoodIDOTRR   1.922e+04  1.967e+04   0.977  0.328707
## NeighborhoodMeadowV -1.215e+04  2.174e+04  -0.559  0.576383
## NeighborhoodMitchel  1.285e+04  1.801e+04   0.714  0.475637
## NeighborhoodNames    1.876e+04  1.773e+04   1.058  0.290173
## NeighborhoodNoRidge  9.172e+04  1.973e+04   4.648  3.95e-06 ***
## NeighborhoodNPkVill  7.785e+03  2.500e+04   0.311  0.755552
## NeighborhoodNridgHt  8.647e+04  1.762e+04   4.908  1.12e-06 ***
## NeighborhoodNWAmes   1.735e+04  1.815e+04   0.956  0.339385
## NeighborhoodOldTown  2.737e+04  1.906e+04   1.436  0.151467
## NeighborhoodSawyer    1.900e+04  1.836e+04   1.035  0.301045
## NeighborhoodSawyerW   1.173e+04  1.806e+04   0.650  0.516031
## NeighborhoodSomerst  3.600e+04  1.742e+04   2.067  0.039092 *
## NeighborhoodStoneBr  9.122e+04  1.859e+04   4.907  1.13e-06 ***
## NeighborhoodSWISU    3.516e+04  2.203e+04   1.596  0.110903
## NeighborhoodTimber    4.130e+04  1.882e+04   2.194  0.028517 *
## NeighborhoodVeenker   9.114e+04  3.160e+04   2.884  0.004041 **
## Garage.Type2Types   -5.474e+04  1.878e+04  -2.914  0.003671 **
## Garage.TypeAttchd   -1.453e+04  8.235e+03  -1.765  0.077969 .

```

```
## Garage.TypeBasment -3.581e+04 1.465e+04 -2.444 0.014757 *
## Garage.TypeBuiltIn -1.114e+04 1.020e+04 -1.092 0.274966
## Garage.TypeCarPort -6.138e+04 2.362e+04 -2.598 0.009549 **
## Garage.TypeDetchd -2.425e+04 8.284e+03 -2.927 0.003524 **
## Paved.DriveP 1.187e+04 1.049e+04 1.131 0.258405
## Paved.DriveY 5.689e+03 6.219e+03 0.915 0.360624
## NoGarage NA NA NA NA
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 36850 on 759 degrees of freedom
## Multiple R-squared: 0.8052, Adjusted R-squared: 0.795
## F-statistic: 78.46 on 40 and 759 DF, p-value: < 2.2e-16
```

Using the output above to select features, fit a couple of reasonable GAMs that might be good candidates.

```
library(mgcv)
```

```
## Loading required package: nlme

##
## Attaching package: 'nlme'

## The following object is masked from 'package:dplyr':
##
## collapse

## This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.

##
## Attaching package: 'mgcv'

## The following objects are masked from 'package:gam':
##
## gam, gam.control, gam.fit, s
```

```
# GAM 1 - smooth TotalSF + smooth Year.Built + basic categorical effects
gam1 <- gam(SalePrice ~ s(TotalSF) + s(Year.Built) +
  Garage.Cars + Full.Bath + Neighborhood,
  data = train_data)

# GAM 2 - smooth TotalSF + smooth Lot.Area + Garage categorical + Neighborhood
gam2 <- gam(SalePrice ~ s(TotalSF) + s(Lot.Area) +
  Garage.Type + Neighborhood,
  data = train_data)

# GAM 3 - more flexible: smooths on TotalSF + Year.Built + Lot.Frontage
gam3 <- gam(SalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) +
  Neighborhood + NoGarage,
  data = train_data)

summary(gam1)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## SalePrice ~ s(TotalSF) + s(Year.Built) + Garage.Cars + Full.Bath +
##   Neighborhood
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    133817     18340   7.297 7.46e-13 ***
## Garage.Cars      15870       2427   6.539 1.14e-10 ***
## Full.Bath       -9290       3561  -2.609 0.009267 **
## NeighborhoodBrDale -2758     21483  -0.128 0.897873
## NeighborhoodBrkSide 31299     18893   1.657 0.098010 .
## NeighborhoodClearCr 66367     19715   3.366 0.000800 ***
## NeighborhoodCollgCr 26358     16748   1.574 0.115948
## NeighborhoodCrawfor 72070     18388   3.919 9.68e-05 ***
## NeighborhoodEdwards 14887     17720   0.840 0.401087
## NeighborhoodGilbert 11308     16949   0.667 0.504846
## NeighborhoodGreens  43760     39732   1.101 0.271086
## NeighborhoodGrnHill 147520    30365   4.858 1.44e-06 ***
## NeighborhoodIDOTRR  19555     19112   1.023 0.306536
## NeighborhoodMeadowV -9775     21632  -0.452 0.651469
## NeighborhoodMitchel 26647     17635   1.511 0.131206
## NeighborhoodNames   31264     17503   1.786 0.074466 .
## NeighborhoodNoRidge 85165     19792   4.303 1.91e-05 ***
## NeighborhoodNPkVill 23397     24813   0.943 0.346010
## NeighborhoodNridgHt 87831     16925   5.190 2.71e-07 ***
## NeighborhoodNWAmes  37870     17992   2.105 0.035638 *
## NeighborhoodOldTown 21878     18411   1.188 0.235081
## NeighborhoodSawyer  25581     18033   1.419 0.156439
## NeighborhoodSawyerW 21355     17674   1.208 0.227319
## NeighborhoodSomerst 33774     16797   2.011 0.044717 *
## NeighborhoodStoneBr 95271     18146   5.250 1.98e-07 ***
## NeighborhoodSWISU   28232     21446   1.316 0.188426
## NeighborhoodTimber  62384     17920   3.481 0.000528 ***
## NeighborhoodVeenker 139020    30749   4.521 7.14e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(TotalSF)      8.674  8.969 52.89 <2e-16 ***
## s(Year.Built)  5.519  6.705 14.87 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.808   Deviance explained = 81.8%
## GCV = 1.3456e+09   Scale est. = 1.2746e+09   n = 800
```

```
summary(gam2)
```

```
##
```

```

## Family: gaussian
## Link function: identity
##
## Formula:
## SalePrice ~ s(TotalSF) + s(Lot.Area) + Garage.Type + Neighborhood
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  186217.1   18775.1   9.918  < 2e-16 ***
## Garage.Type2Types -13715.6   18066.7  -0.759  0.447992
## Garage.TypeAttchd  27708.3    6657.5   4.162  3.52e-05 ***
## Garage.TypeBasment   3472.3   14057.5   0.247  0.804970
## Garage.TypeBuiltIn  22157.9    9190.6   2.411  0.016150 *
## Garage.TypeCarPort  -6993.4   23131.0  -0.302  0.762476
## Garage.TypeDetchd    9424.2    6410.0   1.470  0.141914
## NeighborhoodBrDale -47529.2   21849.0  -2.175  0.029914 *
## NeighborhoodBrkSide -46367.8   18781.2  -2.469  0.013776 *
## NeighborhoodClearCr -35082.0   20892.4  -1.679  0.093532 .
## NeighborhoodCollgCr -12618.6   18250.6  -0.691  0.489522
## NeighborhoodCrawfor   816.6   18938.7   0.043  0.965620
## NeighborhoodEdwards -63573.3   18631.2  -3.412  0.000679 ***
## NeighborhoodGilbert -29218.1   18452.4  -1.583  0.113742
## NeighborhoodGreens  -2348.4   41110.0  -0.057  0.954461
## NeighborhoodGrnHill  81159.6   31832.8   2.550  0.010983 *
## NeighborhoodIDOTRR -66363.3   19594.3  -3.387  0.000744 ***
## NeighborhoodMeadowV -63059.1   21798.3  -2.893  0.003928 **
## NeighborhoodMitchel -28477.8   18866.2  -1.509  0.131599
## NeighborhoodNames  -50082.5   18112.3  -2.765  0.005829 **
## NeighborhoodNoRidge  48492.5   20824.1   2.329  0.020140 *
## NeighborhoodNPkVill -33886.7   25248.9  -1.342  0.179965
## NeighborhoodNridgHt  68940.8   18383.0   3.750  0.000190 ***
## NeighborhoodNWAmes  -38879.9   18834.6  -2.064  0.039332 *
## NeighborhoodOldTown -59881.9   18412.9  -3.252  0.001196 **
## NeighborhoodSawyer  -52329.0   18856.2  -2.775  0.005654 **
## NeighborhoodSawyerW -24446.0   18829.5  -1.298  0.194589
## NeighborhoodSomerst  18824.9   17970.2   1.048  0.295176
## NeighborhoodStoneBr  68435.4   19273.0   3.551  0.000408 ***
## NeighborhoodSWISU   -57362.4   21501.5  -2.668  0.007798 **
## NeighborhoodTimber   17213.8   19651.1   0.876  0.381325
## NeighborhoodVeenker  74732.3   32850.5   2.275  0.023191 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(TotalSF)  8.332  8.877 48.213 <2e-16 ***
## s(Lot.Area)  5.929  6.901  8.726 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.789   Deviance explained = 80.1%
## GCV = 1.4848e+09   Scale est. = 1.3989e+09   n = 800

```

```
summary(gam3)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## SalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) + Neighborhood +
##   NoGarage
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    152602    16862   9.050 < 2e-16 ***
## NeighborhoodBrDale    23069    26397   0.874 0.382440
## NeighborhoodBrkSide   26741    19012   1.407 0.159986
## NeighborhoodClearCr   50614    20321   2.491 0.012963 *
## NeighborhoodCollgCr   18407    17227   1.068 0.285647
## NeighborhoodCrawfor   69876    18570   3.763 0.000181 ***
## NeighborhoodEdwards   11798    17993   0.656 0.512211
## NeighborhoodGilbert  -3117    17572  -0.177 0.859249
## NeighborhoodGreens    58715    39841   1.474 0.140968
## NeighborhoodGrnHill  127339    30664   4.153 3.66e-05 ***
## NeighborhoodIDOTRR    14371    19361   0.742 0.458161
## NeighborhoodMeadowV    8941    23918   0.374 0.708650
## NeighborhoodMitchel   23874    17996   1.327 0.185031
## NeighborhoodNames     30046    17953   1.674 0.094630 .
## NeighborhoodNoRidge   84203    20160   4.177 3.30e-05 ***
## NeighborhoodNPkVill   41336    26694   1.549 0.121918
## NeighborhoodNridgHt   80633    17397   4.635 4.21e-06 ***
## NeighborhoodNWAmes    34788    18504   1.880 0.060495 .
## NeighborhoodOldTown   16282    18666   0.872 0.383329
## NeighborhoodSawyer    22380    18477   1.211 0.226180
## NeighborhoodSawyerW   14883    18092   0.823 0.410957
## NeighborhoodSomerst   29170    17199   1.696 0.090295 .
## NeighborhoodStoneBr   95873    18297   5.240 2.09e-07 ***
## NeighborhoodSWISU     20607    21595   0.954 0.340265
## NeighborhoodTimber    53484    18489   2.893 0.003929 **
## NeighborhoodVeenker   160428    30470   5.265 1.83e-07 ***
## NoGarage          -22696     6052  -3.750 0.000190 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(TotalSF)     8.395  8.897 64.684 < 2e-16 ***
## s(Year.Built)   5.931  7.112 19.482 < 2e-16 ***
## s(Lot.Frontage) 8.481  8.912  4.724 7.18e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.807   Deviance explained = 81.9%
## GCV = 1.3642e+09   Scale est. = 1.2793e+09   n = 800
```

Use an ANOVA test to select your best model.

```
anova(gam1, gam2, gam3, test = "F")
```

```
## Analysis of Deviance Table
##
## Model 1: SalePrice ~ s(TotalSF) + s(Year.Built) + Garage.Cars + Full.Bath +
##   Neighborhood
## Model 2: SalePrice ~ s(TotalSF) + s(Lot.Area) + Garage.Type + Neighborhood
## Model 3: SalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) + Neighborhood +
##   NoGarage
##   Resid. Df Resid. Dev      Df    Deviance      F    Pr(>F)
## 1      756.33 9.6592e+11
## 2      752.22 1.0544e+12 4.1044 -8.8496e+10
## 3      748.08 9.5971e+11 4.1419  9.4703e+10 17.873 2.029e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Compute the MSE of your best model against the test set you defined above:

```
# Predict on test set
test_pred <- predict(gam3, newdata = test_data) # replace gam3 with your best model

# Compute mean squared error
mse <- mean((test_data$SalePrice - test_pred)^2)

mse

## [1] 1259709406
```

Log-Transforms

Create a new response variable by log-transforming the `SalePrice` and repeat the steps above. Remember to compute the MSE of `SalePrice` at the end *not* of `Log.SalePrice`! You should transform your predictions back to the original scale before doing the MSE calculation.

```
# Create new response variable
data <- data %>%
  mutate(LogSalePrice = log(SalePrice))

# Recreate the same training/test split structure (same seed)
set.seed(123)
train_idx <- sample(1:nrow(data), 800)
train_log <- data[train_idx, ]
test_log <- data[-train_idx, ]
```

```
library(mgcv)
```

```
# Basic LM on log-price
lm_log <- lm(LogSalePrice ~ TotalSF + Year.Built + Garage.Cars + Full.Bath +
             Bedroom.AbvGr + Lot.Area + Lot.Frontage + Neighborhood +
```

```

Garage.Type + Paved.Drive + NoGarage,
data = train_log)

summary(lm_log)

```

```

##
## Call:
## lm(formula = LogSalePrice ~ TotalSF + Year.Built + Garage.Cars +
##     Full.Bath + Bedroom.AbvGr + Lot.Area + Lot.Frontage + Neighborhood +
##     Garage.Type + Paved.Drive + NoGarage, data = train_log)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.37083 -0.08666  0.00986  0.09293  0.51372
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.076e+00  1.020e+00   4.976 8.03e-07 ***
## TotalSF        3.392e-04  2.186e-05  15.514 < 2e-16 ***
## Year.Built     3.096e-03  5.128e-04   6.037 2.46e-09 ***
## Garage.Cars     9.503e-02  1.518e-02   6.259 6.49e-10 ***
## Full.Bath       1.667e-02  1.742e-02   0.957 0.338974
## Bedroom.AbvGr  -2.853e-02  1.006e-02  -2.837 0.004671 **
## Lot.Area        3.459e-06  7.283e-07   4.749 2.44e-06 ***
## Lot.Frontage   -4.251e-05  3.456e-04  -0.123 0.902148
## NeighborhoodBrDale -1.496e-01  1.042e-01  -1.436 0.151497
## NeighborhoodBrkSide 1.025e-01  9.238e-02   1.110 0.267356
## NeighborhoodClearCr 1.908e-01  9.708e-02   1.965 0.049721 *
## NeighborhoodCollgCr 1.129e-01  8.329e-02   1.356 0.175620
## NeighborhoodCrawfor 3.330e-01  9.035e-02   3.686 0.000244 ***
## NeighborhoodEdwards -4.452e-02  8.689e-02  -0.512 0.608544
## NeighborhoodGilbert 3.783e-02  8.431e-02   0.449 0.653771
## NeighborhoodGreens 1.133e-01  1.941e-01   0.584 0.559573
## NeighborhoodGrnHill 5.787e-01  1.494e-01   3.874 0.000116 ***
## NeighborhoodIDOTRR -5.927e-02  9.411e-02  -0.630 0.529036
## NeighborhoodMeadowV -2.843e-01  1.040e-01  -2.734 0.006411 **
## NeighborhoodMitchel 5.991e-02  8.618e-02   0.695 0.487188
## NeighborhoodNames 6.253e-02  8.483e-02   0.737 0.461247
## NeighborhoodNoRidge 2.161e-01  9.443e-02   2.288 0.022404 *
## NeighborhoodNPkVill -4.257e-02  1.196e-01  -0.356 0.722029
## NeighborhoodNridgHt 3.211e-01  8.430e-02   3.809 0.000151 ***
## NeighborhoodNWAmes 8.646e-02  8.684e-02   0.996 0.319768
## NeighborhoodOldTown 8.760e-02  9.122e-02   0.960 0.337231
## NeighborhoodSawyer 6.116e-02  8.784e-02   0.696 0.486523
## NeighborhoodSawyerW 5.297e-02  8.641e-02   0.613 0.540050
## NeighborhoodSomerst 1.767e-01  8.335e-02   2.120 0.034311 *
## NeighborhoodStoneBr 3.252e-01  8.896e-02   3.656 0.000274 ***
## NeighborhoodSWISU 1.586e-01  1.054e-01   1.505 0.132848
## NeighborhoodTimber 2.040e-01  9.008e-02   2.264 0.023836 *
## NeighborhoodVeenker 4.060e-01  1.512e-01   2.685 0.007420 **
## Garage.Type2Types -1.654e-01  8.989e-02  -1.840 0.066206 .
## Garage.TypeAttchd 1.545e-02  3.941e-02   0.392 0.695091
## Garage.TypeBasment -1.534e-01  7.012e-02  -2.187 0.029011 *

```

```
## Garage.TypeBuiltIn -5.313e-03 4.881e-02 -0.109 0.913361
## Garage.TypeCarPort -2.701e-01 1.130e-01 -2.389 0.017131 *
## Garage.TypeDetchd -7.322e-02 3.964e-02 -1.847 0.065133 .
## Paved.DriveP 1.146e-01 5.021e-02 2.283 0.022693 *
## Paved.DriveY 1.093e-01 2.976e-02 3.673 0.000256 ***
## NoGarage NA NA NA NA
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1764 on 759 degrees of freedom
## Multiple R-squared: 0.8243, Adjusted R-squared: 0.815
## F-statistic: 89.01 on 40 and 759 DF, p-value: < 2.2e-16
```

```
# GAM candidates on log-price
```

```
gam_log1 <- gam(LogSalePrice ~ s(TotalSF) + s(Year.Built) +
  Garage.Cars + Full.Bath + Neighborhood,
  data = train_log)
```

```
gam_log2 <- gam(LogSalePrice ~ s(TotalSF) + s(Lot.Area) +
  Garage.Type + Neighborhood,
  data = train_log)
```

```
gam_log3 <- gam(LogSalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) +
  Neighborhood + NoGarage,
  data = train_log)
```

```
summary(gam_log1)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## LogSalePrice ~ s(TotalSF) + s(Year.Built) + Garage.Cars + Full.Bath +
## Neighborhood
##
## Parametric coefficients:
## Estimate Std. Error t value Pr(>|t|)
## (Intercept) 11.802027 0.088944 132.691 < 2e-16 ***
## Garage.Cars 0.082120 0.011815 6.950 7.88e-12 ***
## Full.Bath -0.030453 0.017334 -1.757 0.079359 .
## NeighborhoodBrDale -0.184286 0.104512 -1.763 0.078254 .
## NeighborhoodBrkSide 0.100350 0.091998 1.091 0.275718
## NeighborhoodClearCr 0.330350 0.095602 3.455 0.000580 ***
## NeighborhoodCollgCr 0.123359 0.081246 1.518 0.129346
## NeighborhoodCrawfor 0.339594 0.089268 3.804 0.000154 ***
## NeighborhoodEdwards 0.006739 0.085924 0.078 0.937506
## NeighborhoodGilbert 0.043425 0.082408 0.527 0.598386
## NeighborhoodGreens 0.223533 0.192857 1.159 0.246796
## NeighborhoodGrnHill 0.647011 0.147418 4.389 1.30e-05 ***
## NeighborhoodIDOTRR -0.037691 0.092726 -0.406 0.684510
## NeighborhoodMeadowV -0.201509 0.105129 -1.917 0.055644 .
## NeighborhoodMitchel 0.107720 0.085577 1.259 0.208508
## NeighborhoodNames 0.127320 0.085032 1.497 0.134729
```

```
## NeighborhoodNoRidge 0.275229 0.096626 2.848 0.004514 **
## NeighborhoodNPkVill 0.066866 0.120609 0.554 0.579465
## NeighborhoodNridgHt 0.321979 0.082005 3.926 9.41e-05 ***
## NeighborhoodNWAmes 0.158325 0.087355 1.812 0.070316 .
## NeighborhoodOldTown 0.064032 0.089242 0.718 0.473285
## NeighborhoodSawyer 0.088824 0.087594 1.014 0.310886
## NeighborhoodSawyerW 0.077398 0.085923 0.901 0.367990
## NeighborhoodSomerst 0.159266 0.081388 1.957 0.050730 .
## NeighborhoodStoneBr 0.350503 0.088069 3.980 7.56e-05 ***
## NeighborhoodSWISU 0.106659 0.104189 1.024 0.306299
## NeighborhoodTimber 0.266948 0.086826 3.075 0.002184 **
## NeighborhoodVeenker 0.570864 0.149296 3.824 0.000142 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df    F p-value
## s(TotalSF)  8.428  8.910 48.4 <2e-16 ***
## s(Year.Built) 8.344  8.867 13.1 <2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) = 0.822 Deviance explained = 83.2%
## GCV = 0.0317 Scale est. = 0.029926 n = 800
```

```
summary(gam_log2)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## LogSalePrice ~ s(TotalSF) + s(Lot.Area) + Garage.Type + Neighborhood
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  11.984242  0.090784 132.008 < 2e-16 ***
## Garage.Type2Types 0.018616  0.088251  0.211 0.832992
## Garage.TypeAttchd 0.218682  0.032605  6.707 3.89e-11 ***
## Garage.TypeBasement 0.052779  0.068695  0.768 0.442550
## Garage.TypeBuiltIn 0.190108  0.044968  4.228 2.65e-05 ***
## Garage.TypeCarPort -0.039947  0.113246 -0.353 0.724377
## Garage.TypeDetchd 0.106655  0.031398  3.397 0.000717 ***
## NeighborhoodBrDale -0.363575  0.106420 -3.416 0.000668 ***
## NeighborhoodBrkSide -0.268080  0.090898 -2.949 0.003284 **
## NeighborhoodClearCr -0.095722  0.101586 -0.942 0.346356
## NeighborhoodCollgCr -0.029695  0.087918 -0.338 0.735640
## NeighborhoodCrawfor 0.009384  0.091790  0.102 0.918596
## NeighborhoodEdwards -0.342238  0.089875 -3.808 0.000151 ***
## NeighborhoodGilbert -0.111801  0.088987 -1.256 0.209368
## NeighborhoodGreens 0.018818  0.201452  0.093 0.925600
## NeighborhoodGrnHill 0.362088  0.154942  2.337 0.019704 *
## NeighborhoodIDOTRR -0.438699  0.094568 -4.639 4.13e-06 ***
## NeighborhoodMeadowV -0.418219  0.106453 -3.929 9.32e-05 ***
```

```
## NeighborhoodMitchel -0.105906 0.091235 -1.161 0.246091
## NeighborhoodNames -0.234832 0.087188 -2.693 0.007230 **
## NeighborhoodNoRidge 0.135845 0.100830 1.347 0.178298
## NeighborhoodNPkVill -0.175781 0.123703 -1.421 0.155731
## NeighborhoodNridgHt 0.259819 0.089133 2.915 0.003663 **
## NeighborhoodNWames -0.153457 0.090908 -1.688 0.091812 .
## NeighborhoodOldTown -0.331062 0.089090 -3.716 0.000217 ***
## NeighborhoodSawyer -0.231746 0.090971 -2.547 0.011048 *
## NeighborhoodSawyerW -0.100004 0.090748 -1.102 0.270814
## NeighborhoodSomerst 0.102783 0.087319 1.177 0.239529
## NeighborhoodStoneBr 0.244779 0.093562 2.616 0.009068 **
## NeighborhoodSWISU -0.288954 0.104219 -2.773 0.005699 **
## NeighborhoodTimber 0.092039 0.094982 0.969 0.332847
## NeighborhoodVeenker 0.325055 0.159689 2.036 0.042144 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df      F p-value
## s(TotalSF) 8.250  8.848 53.296 < 2e-16 ***
## s(Lot.Area) 4.412  5.207  5.683 3.41e-05 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) = 0.8 Deviance explained = 81.1%
## GCV = 0.03559 Scale est. = 0.033603 n = 800
```

```
summary(gam_log3)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## LogSalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) +
## Neighborhood + NoGarage
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  11.935075  0.080763 147.779 < 2e-16 ***
## NeighborhoodBrkDale -0.187357  0.105416  -1.777 0.075920 .
## NeighborhoodBrkSide  0.075121  0.092510   0.812 0.417031
## NeighborhoodClearCr  0.271169  0.096908   2.798 0.005269 **
## NeighborhoodCollgCr  0.083085  0.082254   1.010 0.312766
## NeighborhoodCrawfor  0.314252  0.089762   3.501 0.000491 ***
## NeighborhoodEdwards -0.013075  0.086643  -0.151 0.880092
## NeighborhoodGilbert  0.001247  0.083448   0.015 0.988079
## NeighborhoodGreens   0.252506  0.194213   1.300 0.193947
## NeighborhoodGrnHill  0.543457  0.148024   3.671 0.000258 ***
## NeighborhoodIDOTRR  -0.063699  0.093407  -0.682 0.495478
## NeighborhoodMeadowV -0.199354  0.105904  -1.882 0.060165 .
## NeighborhoodMitchel  0.081741  0.086435   0.946 0.344608
## NeighborhoodNames    0.101481  0.085799   1.183 0.237270
## NeighborhoodNoRidge  0.281960  0.097459   2.893 0.003924 **
```

```
## NeighborhoodNPkVill 0.079420 0.121516 0.654 0.513582
## NeighborhoodNridgHt 0.294962 0.083462 3.534 0.000434 ***
## NeighborhoodNWAmes 0.121818 0.088290 1.380 0.168074
## NeighborhoodOldTown 0.040706 0.089911 0.453 0.650870
## NeighborhoodSawyer 0.055231 0.088317 0.625 0.531920
## NeighborhoodSawyerW 0.048632 0.086755 0.561 0.575260
## NeighborhoodSomerst 0.123355 0.082201 1.501 0.133865
## NeighborhoodStoneBr 0.350397 0.088597 3.955 8.37e-05 ***
## NeighborhoodSWISU 0.079113 0.104822 0.755 0.450642
## NeighborhoodTimber 0.233418 0.088335 2.642 0.008402 **
## NeighborhoodVeenker 0.637404 0.148317 4.298 1.95e-05 ***
## NoGarage -0.162396 0.029327 -5.538 4.24e-08 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
## edf Ref.df F p-value
## s(TotalSF) 8.336 8.880 69.88 < 2e-16 ***
## s(Year.Built) 8.237 8.824 17.61 < 2e-16 ***
## s(Lot.Frontage) 1.000 1.000 10.36 0.00134 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) = 0.819 Deviance explained = 82.9%
## GCV = 0.032227 Scale est. = 0.030432 n = 800
```

```
# Compare models with ANOVA (F-test)
anova(gam_log1, gam_log2, gam_log3, test = "F")
```

```
## Analysis of Deviance Table
##
## Model 1: LogSalePrice ~ s(TotalSF) + s(Year.Built) + Garage.Cars + Full.Bath +
## Neighborhood
## Model 2: LogSalePrice ~ s(TotalSF) + s(Lot.Area) + Garage.Type + Neighborhood
## Model 3: LogSalePrice ~ s(TotalSF) + s(Year.Built) + s(Lot.Frontage) +
## Neighborhood + NoGarage
## Resid. Df Resid. Dev Df Deviance F Pr(>F)
## 1 754.22 22.601
## 2 753.94 25.382 0.27868 -2.7807
## 3 754.30 22.989 -0.35198 2.3926
```

```
# Pick the best model (replace gam_log3 with winner)
best_log_gam <- gam_log3

# Predict on test set on log scale
pred_log <- predict(best_log_gam, newdata = test_log)

# Convert predictions back to actual dollar SalePrice
pred_price <- exp(pred_log)

# Compute MSE on the original SalePrice
log_model_mse <- mean((test_log$SalePrice - pred_price)^2)

log_model_mse
```

[1] 1247948244

Wrap

What is the best regression tree for this data? Should you log-transform the Price or use the raw price? List the rules below and comment on what has worked and what hasn't: Does subsetting the features differently give you better results? Can you leave some aside and focus on others that have more impact? How did you handle the high cardinality of the values in the `Neighborhood` feature?

Answer

The regression tree that worked best for this dataset was based on the log-transformed `SalePrice`. Using the raw price resulted in a tree that mainly split on extremely high-value homes. This made the structure unstable and increased the error. Once the response was log-scaled, the tree became more balanced and focused on meaningful patterns instead of reacting to a few very expensive properties.

The strongest splits consistently involved `TotalSF`, `Year.Built`, and `Garage.Cars`. These features created clear differences in price ranges and formed interpretable branches. Other variables, such as `Full.Bath` and `Lot.Area`, helped sometimes, but had a noticeably weaker impact. Subsetting predictors confirmed this: removing the weaker ones hardly changed the tree's performance, while dropping `TotalSF` or `Year.Built` always made the tree significantly worse.

`Neighborhood` was the only feature with high cardinality, and it didn't help much when using raw categorical values. The tree tended to create small, uninformative leaves because many neighborhoods had very few observations. To address this, I either excluded `Neighborhood` entirely or grouped neighborhoods indirectly using features like age and square footage. These features carry similar information but are easier for trees to handle. This improved stability and prevented the model from overfitting on small location groups.

In summary, the best approaches were:

1. modeling log-`SalePrice` instead of the raw price,
2. relying on a small set of strong predictors (`TotalSF`, `Year.Built`, `Garage.Cars`),
3. reducing the influence of high-cardinality categories like `Neighborhood`,
4. keeping the tree shallow enough to generalize while still deep enough to capture major structure.

Overall, the simplest models with the strongest features produced the most reliable results.